SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: **MLA03**

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO1: *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems.(BL2,BL3)*

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

QUESTIONS: 10 **Total Marks: 50**

Annexure A

CODING CHALLENGE

Duration: 2hrs

NAME:-M.GUNA SHEKAR

REG.NO:-192472345

1.RL Framework – Agent–Environment Interaction

Problem Statement

Design and understand a Reinforcement Learning framework in which an agent interacts with an environment by observing states, selecting actions, and receiving rewards. Study how cumulative rewards guide the agent toward better decisions.

Objectives

- Understand agent–environment interaction.
- Identify states, actions, and rewards.
- Understand how rewards influence learning.
- Explain the importance of cumulative reward in decision-making.

Eligibility

- Basic knowledge of Reinforcement Learning.
- Basic understanding of Python programming.
- Familiarity with states, actions, and rewards.

Challenge Rules

- The agent must observe the current state before taking an action.
- Each action must produce a reward from the environment.
- The agent should try to maximize cumulative reward.
- The solution should demonstrate the RL interaction clearly.

CODE:

```
import numpy as np

states = [0, 1, 2, 3, 4]

actions = ["left", "right"]

rewards = {

  0: {"left": -1, "right": 0},

  1: {"left": 0, "right": 1},

  2: {"left": 0, "right": 2},

  3: {"left": 1, "right": 5},

  4: {"left": 2, "right": 10}

}
```

```
state = 0
total_reward = 0
for step in range(5):
    action = max(rewards[state], key=rewards[state].get)
    reward = rewards[state][action]
    print("State:", state)
    print("Action:", action)
    print("Reward:", reward)
    total_reward += reward
    if action == "right":
        state = min(state + 1, 4)
    else:
        state = max(state - 1, 0)
print("\nCumulative Reward:", total_reward)-
```

Environment Setup

- Python 3.x
- NumPy
- Matplotlib
- Gymnasium (optional)

Input Format

- Initial environment state.
- Available actions.
- Reward values generated by the environment.
- Number of interaction steps/episodes.

Output Format

Step: 1

State: 1

Action: right

Reward: 1

Step: 2

State: 2

Action: right

Reward: 1

Total Reward: 5**Constraints**

- The agent can select only valid actions.
- The agent must operate within the defined environment.
- Rewards should be accumulated over the episode.
- The objective is to maximize cumulative reward.

2. Modeling Decision-Making Using MDP

Problem Statement

Model a decision-making problem as a Markov Decision Process (MDP) by defining states, actions, transition probabilities, rewards, and a discount factor.

Objectives

- Understand the concept of MDP.
- Define the action space.
- Model transition probabilities.
- Define a reward function.
- Understand the role of the discount factor.

Eligibility

- Basic knowledge of probability.
- Understanding of Reinforcement Learning.
- Basic Python programming knowledge.

Challenge Rules

- Define all states clearly.
- Define valid actions for each state.
- Transition probabilities must be valid.
- Rewards must be assigned to state transitions.
- The discount factor must lie between 0 and 1.

CODE:-

```
states = ["A", "B", "C"]

actions = ["Left", "Right"]

rewards = {

    "A": 1,
```

"B": 5,

"C": 10

}

discount = 0.9

print("States:", states)

print("Actions:", actions)

print("Rewards:")

for state in states:

print(state, "=", rewards[state])

print("Discount Factor:", discount)Environment Setup

- Python 3.x
- NumPy
- Matplotlib (optional)

Output Format

States: ['A', 'B', 'C']

Actions: ['Left', 'Right']

Rewards:

A = 1

B = 5

C = 10

Discount Factor: 0.9

Constraints

- $0 \leq P(s'|s,a) \leq 1$
- Sum of transition probabilities for a state-action pair must equal 1.
- $0 \leq \gamma < 1$
- Only valid actions can be selected.

3. Policy and Value Functions Using Bellman Equation

Problem Statement

Develop an understanding of policy and value functions in Reinforcement Learning and use the Bellman Equation to evaluate the long-term benefits of states and actions.

Objectives

- Understand policy functions.
- Understand state-value functions.
- Understand action-value functions.
- Apply the Bellman Equation.
- Evaluate long-term rewards.

Eligibility

- Basic knowledge of RL and MDP.
- Understanding of mathematical equations.
- Basic Python programming knowledge.

Challenge Rules

- Define a valid policy.
- Calculate state-value values.
- Calculate action-value values.
- Use the discount factor correctly.
- Compare actions based on their expected long-term rewards.

Programming Languages

CODE:-

```
reward = 10

discount = 0.9

value = reward + discount * reward
```

```
print("Reward:", reward)

print("Discount:", discount)

print("State Value:", value)

if value > 10:

    print("Best Action: Right")

else:

    print("Best Action: Left")
```

Environment Setup

- Python 3.x
- NumPy
- Matplotlib (optional)

Output Format

Reward: 10

Discount: 0.9

State Value: 19.0

Best Action: Right

Constraints

- Values must be calculated using valid states and actions.
- Discount factor must satisfy $0 \leq \gamma < 1$.
- Only available actions can be evaluated.
- The selected action should maximize expected long-term reward.

4. Exploration–Exploitation Trade-off

Problem Statement

Implement and compare exploration strategies such as **ε-greedy** and **Softmax** to understand how an RL agent balances trying new actions with selecting actions that are already known to provide high rewards.

Objectives

- Understand exploration and exploitation.
- Implement ε-greedy strategy.
- Understand Softmax action selection.
- Compare learning efficiency.
- Identify suitable action-selection strategies.

Eligibility

- Basic Reinforcement Learning knowledge.
- Basic probability knowledge.
- Python programming skills.

Challenge Rules

- The agent must explore different actions.
- The agent should exploit high-reward actions.
- ε-greedy must use a valid epsilon value.
- Softmax must assign probabilities according to action values.
- The strategies should be compared using rewards.

Programming Languages

- Python:-
- CODE:-

```
import random

epsilon = 0.2
```

```
actions = ["Left", "Right"]

for i in range(5):

    if random.random() < epsilon:

            action = random.choice(actions)

        print("Exploration:", action)

else:

        action = "Right"

        print("Exploitation:", action)
```

Environment Setup

- Python 3.x
- NumPy
- Matplotlib

Output Format

Exploitation: Right

Exploitation: Right

Exploration: Left

Exploitation: Right

Exploitation: Right

Constraints

- $0 \leq \varepsilon \leq 1$
- Temperature must be greater than 0.
- Only available actions can be selected.
- The agent should balance exploration and exploitation.

5. Reward Shaping in Reinforcement Learning

Problem Statement

Design a reward-shaping mechanism that provides additional intermediate rewards to an RL agent to accelerate learning and guide it toward the desired goal while avoiding suboptimal policies.

Objectives

- Understand reward shaping.
- Accelerate the learning process.
- Provide useful intermediate rewards.
- Reduce unnecessary exploration.
- Prevent the agent from learning suboptimal policies.

Eligibility

- Basic knowledge of Reinforcement Learning.
- Understanding of rewards and policies.
- Basic Python programming knowledge.

Challenge Rules

- Positive rewards should encourage desirable actions.
- Negative rewards should discourage undesirable actions.
- Rewards should guide the agent toward the final goal.
- Reward shaping should not encourage incorrect shortcuts.
- The final policy should maximize the intended objective.

Programming Languages

position = 0

goal = 5

total_reward = 0

while position < goal:

```
    position = position + 1
 if position == goal:
        reward = 10
    else:
        reward = 1
    total_reward = total_reward + reward
    print("Position:", position)
    print("Reward:", reward)
print("Goal Reached!")
print("Total Reward:", total_reward)
```

Environment Setup

- Python 3.x
- NumPy
- Matplotlib
- Gymnasium (optional)

Output Format

Position: 1

Reward: 1

Position: 2

Reward: 1

Position: 3

Reward: 1

Position: 4

Reward: 1

Position: 5

Reward: 10

Goal Reached!

Total Reward: 14

Constraints

- Reward values must be carefully selected.
- Excessive positive rewards should not encourage undesirable behavior.
- Negative rewards should discourage unwanted actions.
- The agent should eventually learn the optimal or near-optimal policy.
- Reward shaping should support the original task objective.